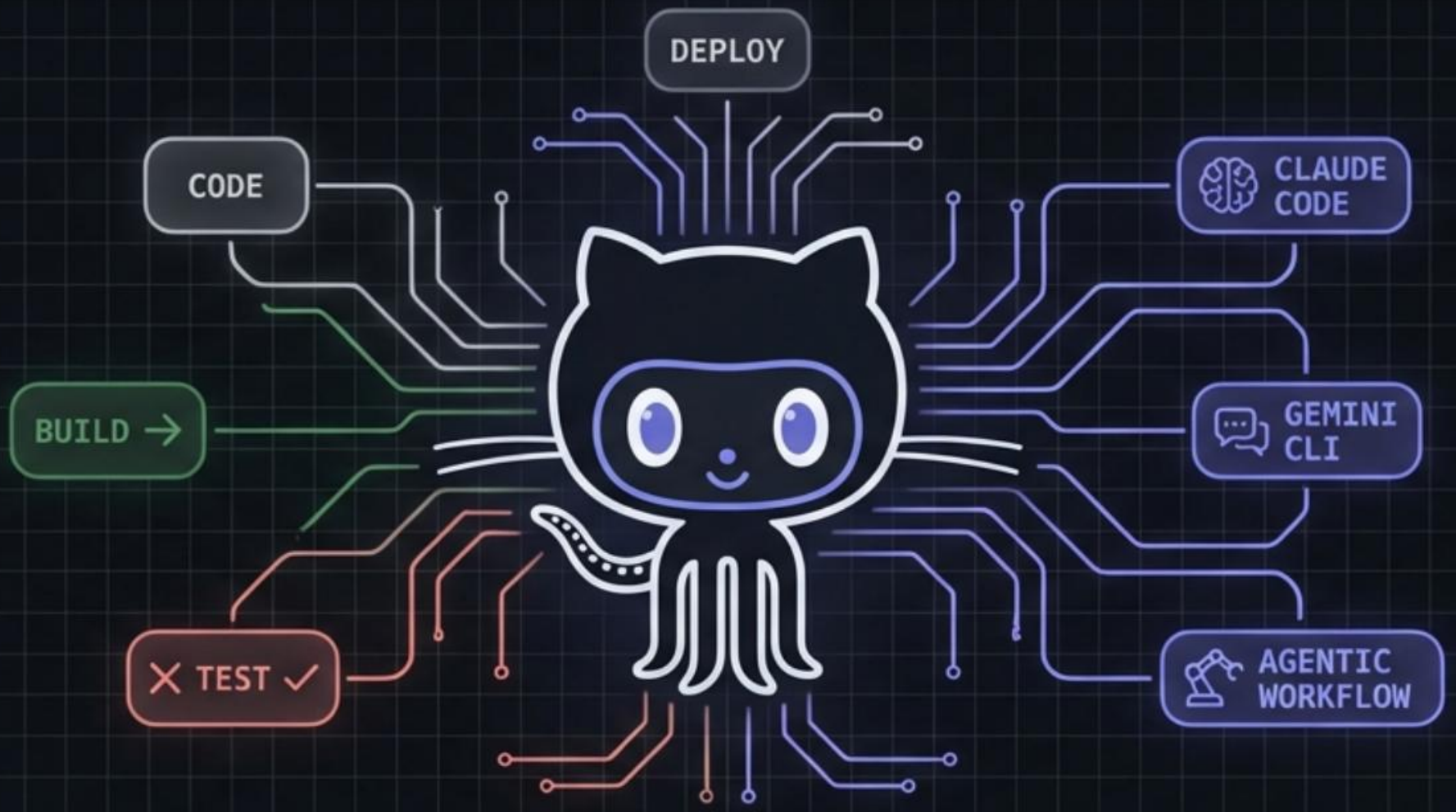
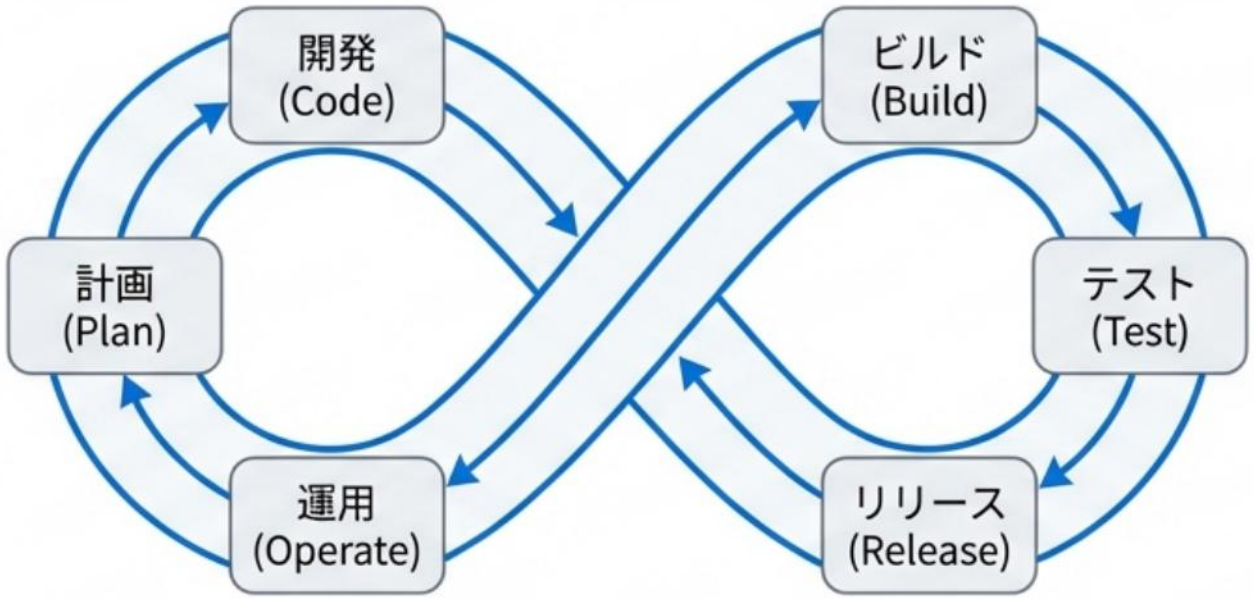


GitHub ActionsとAIエージェント

開発の持続可能性とCI/CDの役割

CI/CDの定義

「ビルド」から「リリース」までを自動化する仕組み。



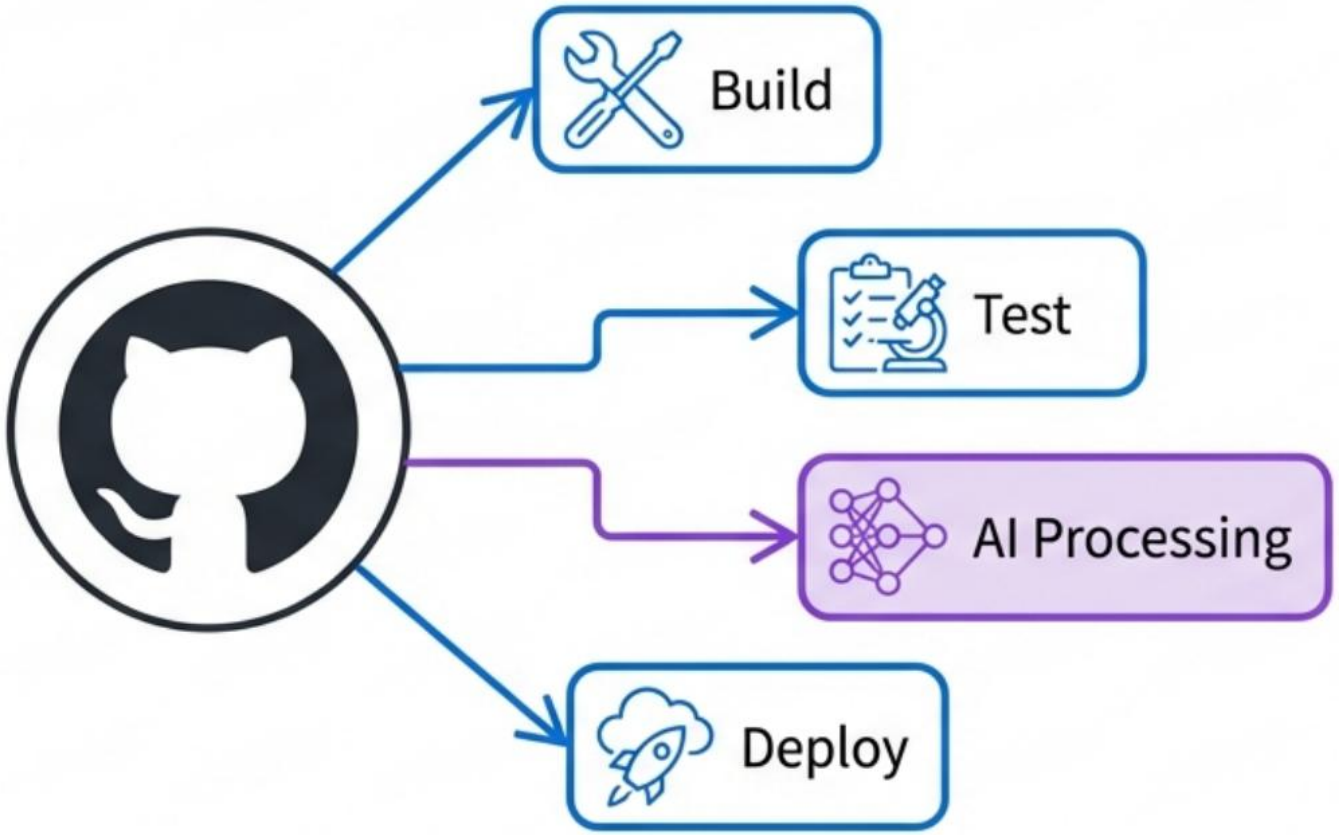
目的 (Why?)

ヒューマンエラーの削減：リリースはエラーが発生しやすいプロセス。自動化で繰り返しに強くなる。

高速なフィードバック：バグを早期発見し、修正コストを下げる。

ビジネスチャンスの最大化：タイムリーな機能提供により競争優位性を築く。

GitHub Actionsとは何か？

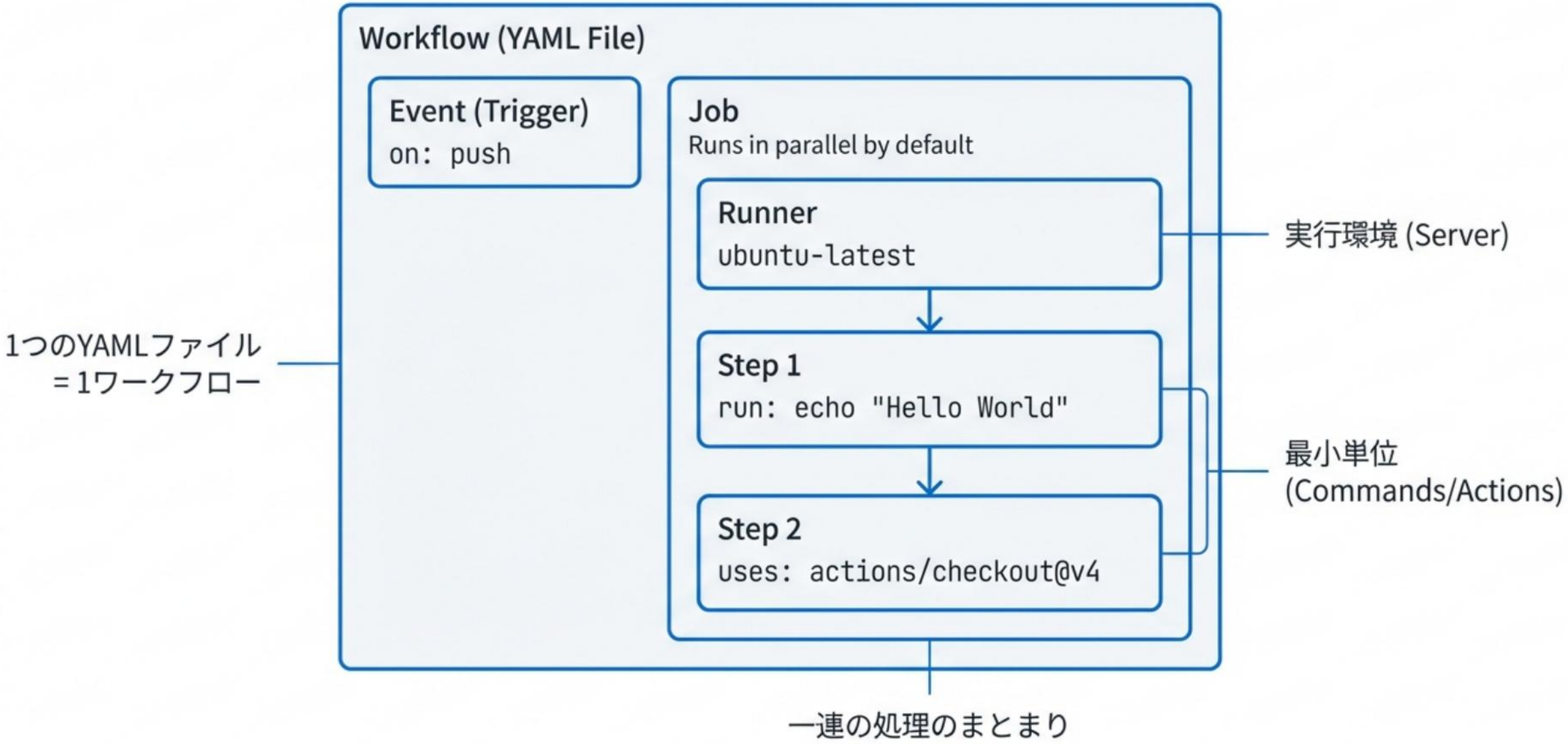


主なトリガー (Events):

- `push`: コードがプッシュされた時
- `pull_request`: PRが作成・更新された時
- `issues`: Issueが作成された時（※AI活用の重要なトリガー）
- `schedule`: 定期実行（cron形式）

GitHubが提供する汎用的なワークフローエンジン。
リポジトリ内のイベントをトリガーにして、任意のタスクを自動実行します。

ワークフローの解剖図：構造と階層



実行環境：ランナーの種類と特性

GitHub-Hosted Runners

GitHubが管理するマネージド環境。手間がかからず、すぐに利用可能。

Linux

Windows

macOS

Ephemeral (エフェメラル)

ジョブごとに起動し、終了時に破棄される『使い捨て』環境。常にクリーンな状態。

Self-Hosted Runners

ユーザー自身のサーバーで実行。スペックやOSを自由にカスタマイズ可能。

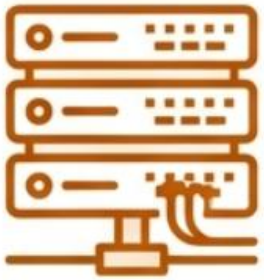
```
runs-on: ubuntu-latest
```

同時に発生した2つの大きな変化

GitHub-Hosted Runners
標準ランナー

大幅値下げ

Linuxランナーを中心に最大39%の価格改定



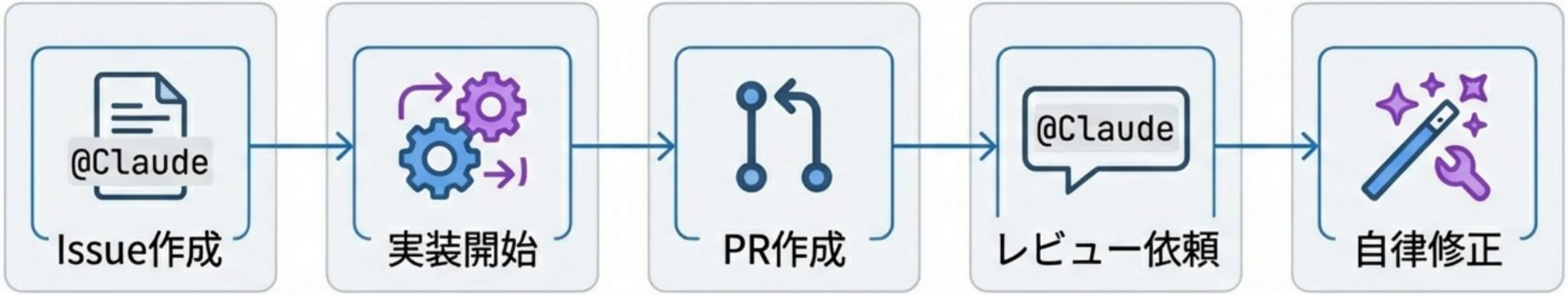
Self-Hosted Runners
セルフホスト

実質的な課金開始

従来「無料」だった実行環境にコストが発生

ユーザーをマネージド環境へ誘導する明確なメッセージ

Case Study 1: Claude Codeによる自律実装

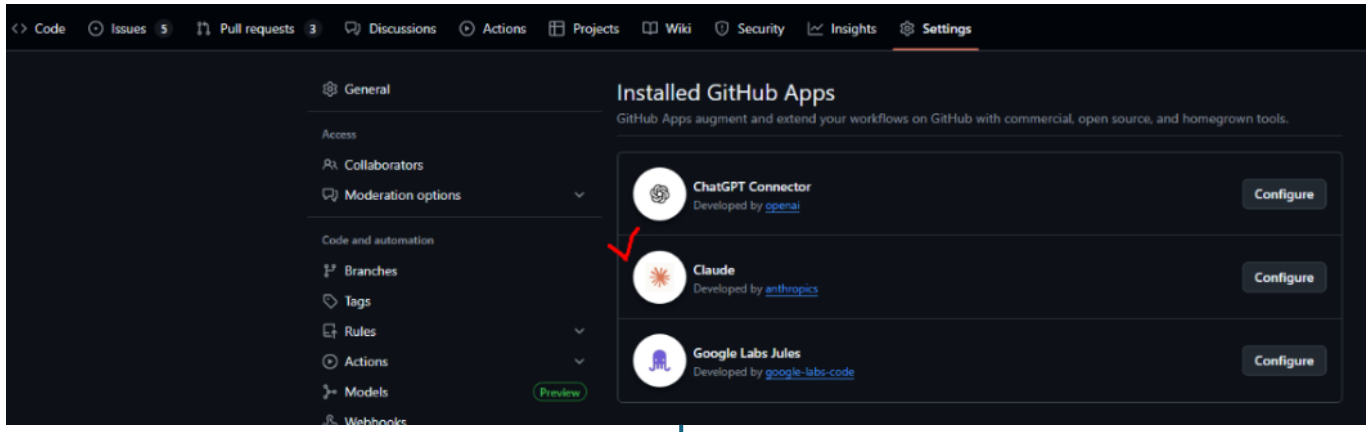


Issue作成から実装、修正までを全自動化する「Agency Model」

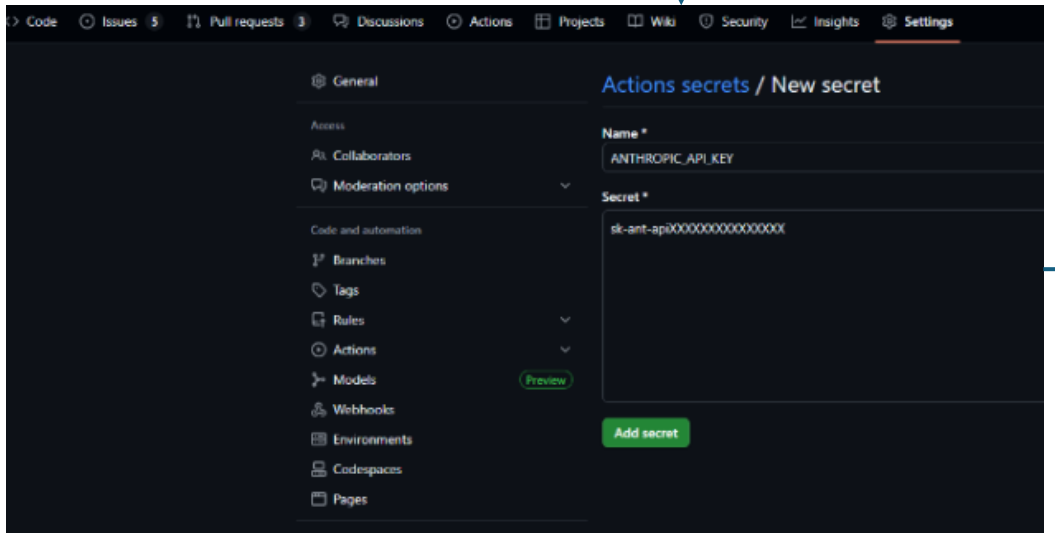
開発者は @Claude とメンションするだけで、AIがエンジニアとして振る舞う。

- **Role**: Junior Engineer / AI Partner
- **Input**: Issue Description
- **Output**: Code & Pull Request

ステップ1：Claude GitHub App のインストール

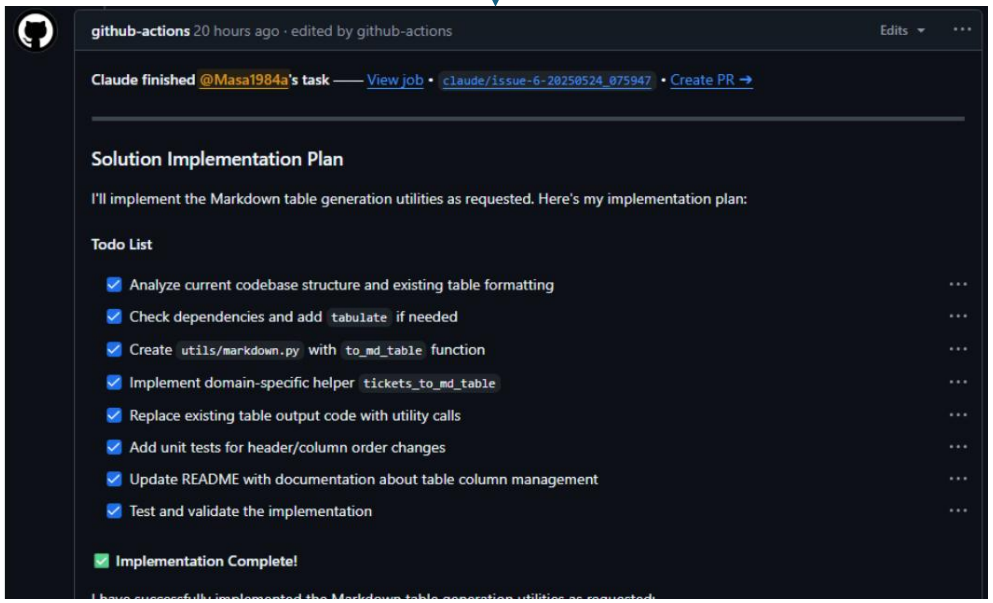


ステップ2：APIキーの登録

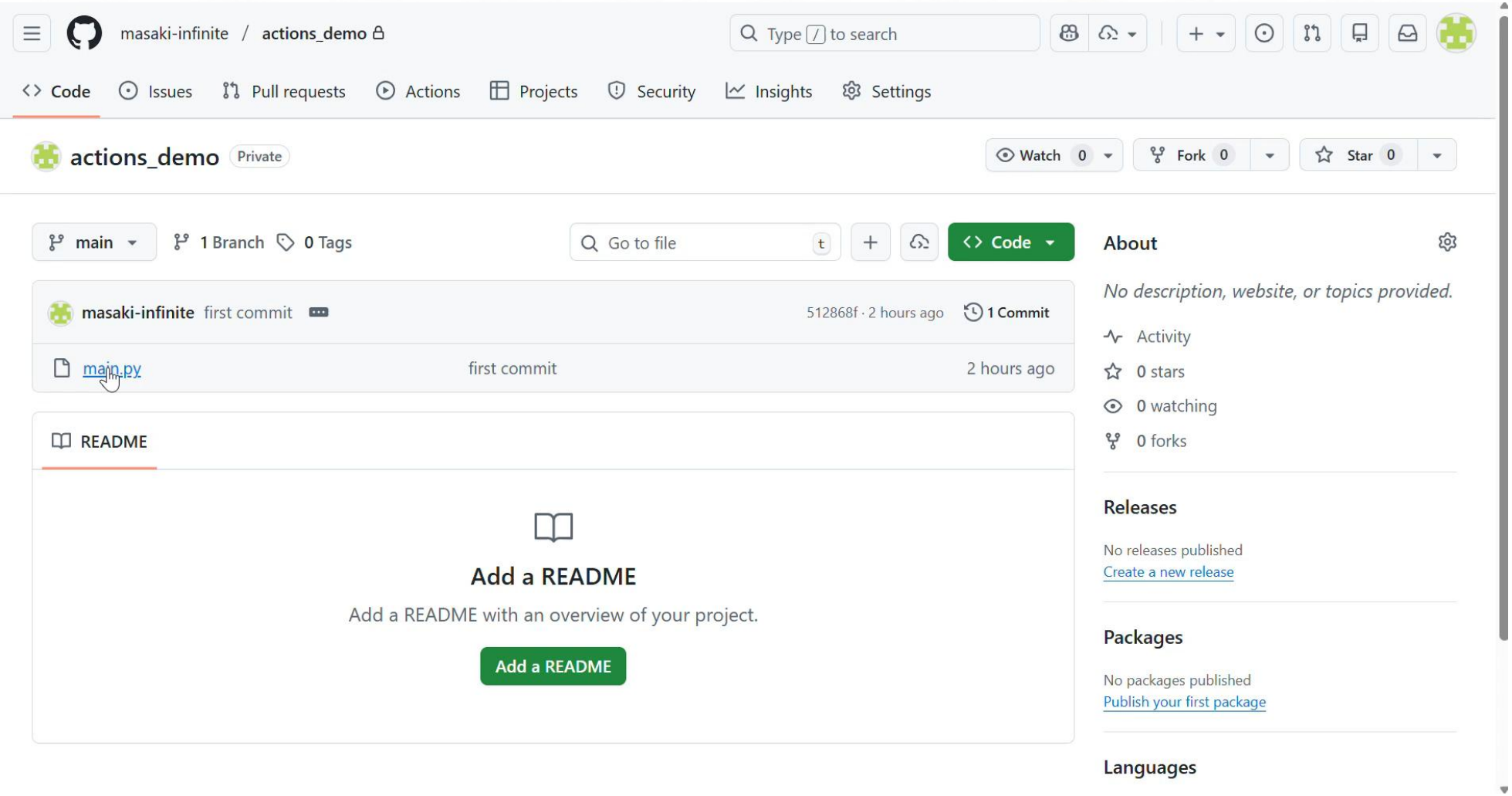
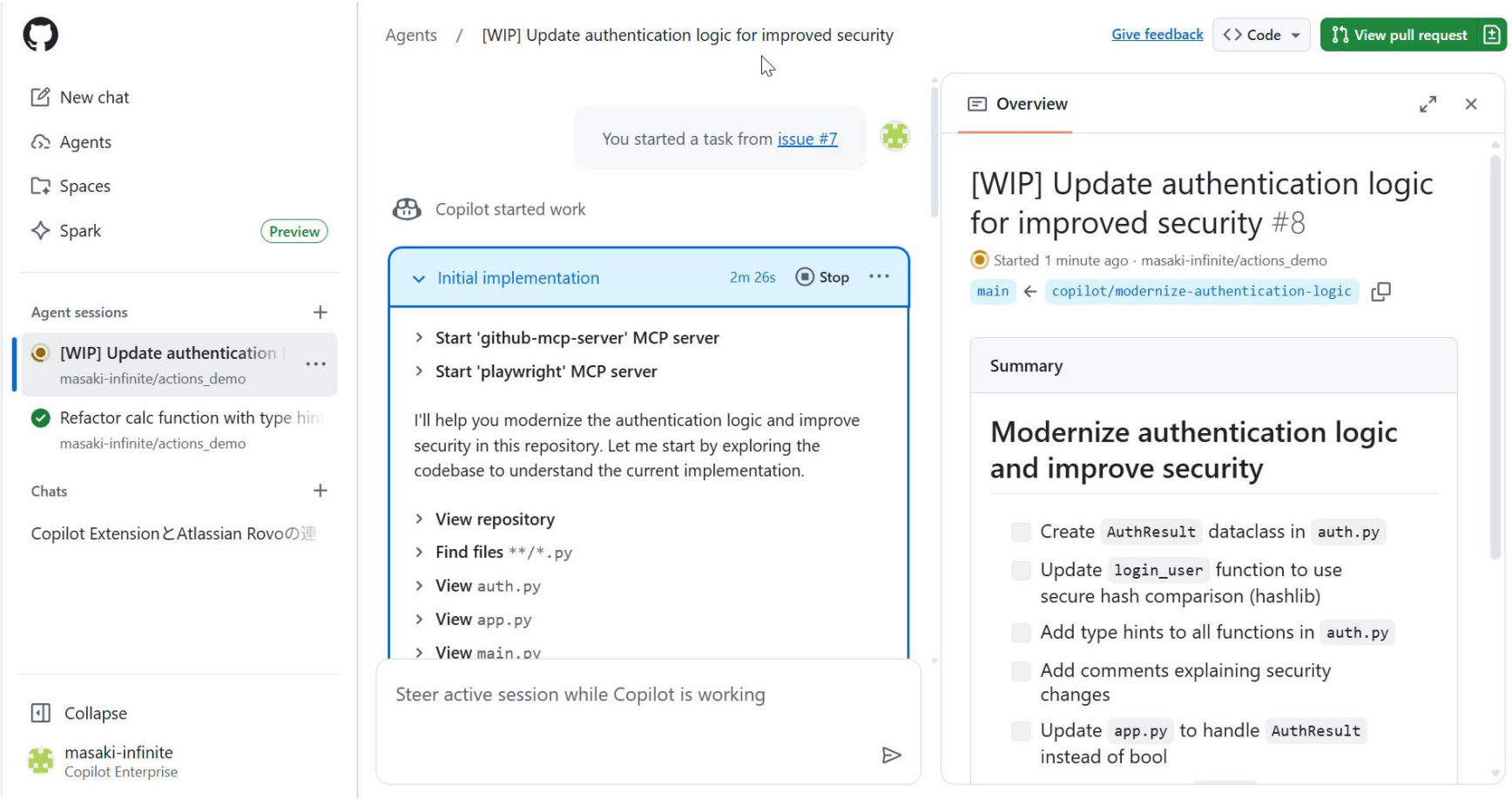


ステップ3：権限の設定

ステップ4：ワークフローファイルの配置



Case Study 2 ：GitHub Copilotによる実装



Mission Control